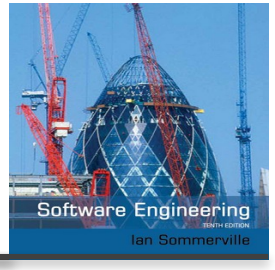
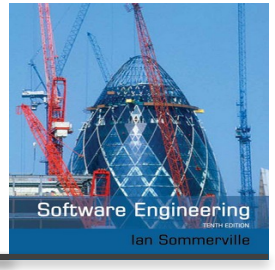
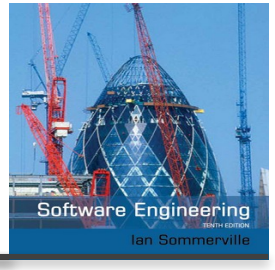


Chapter 2 – Software Processes

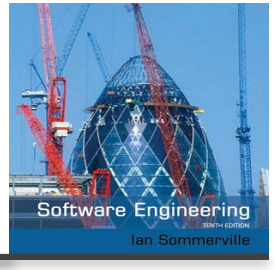
The software process



- **Software process**: A structured set of activities required to develop a software system.
- **All software processes should involve**:
  - **Specification** – defining what the system should do;
  - **Design** – designing what the system should do;
  - **implementation** – defining the organization of the system and implementing the system;
  - **Validation** – checking that it does what the customer wants;
  - **Evolution** – changing the system in response to changing customer needs.
- A software process model is an abstract representation of a process.
  - It **presents a description of a process from some perspective**.

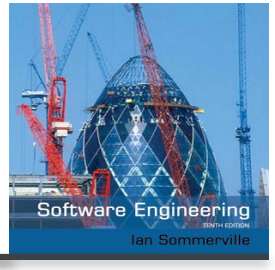


- The Process **activities** involve:
  - **specifying a data model**
    - **For example,**
      - Hierarchical Data Model.
      - Relational Data Model.
      - Entity-relationship (ER) Data Model.
      - Object-oriented Data Model.
      - Dimensional Data Model.
  - **designing a user interface.(layouts' protoype)**
  - **ordering of the activities**.

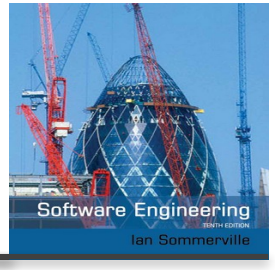


- Process descriptions include:
  - **Products**,
    - outcomes of a process activity;
  - **Roles**,
    - responsibilities of the people involved in the process;
  - **Pre- and post-conditions**,
    - which are statements that are true before and after a process activity has been enacted or a product produced.
    - Example authorization module's pre and post conditions.

Software processes

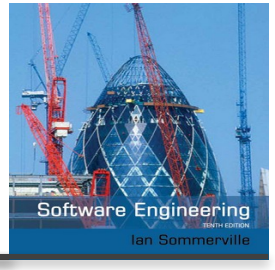


- **Plan-driven processes:**
  - all the process activities are planned
  - progress is measured against this plan.
- **agile processes**
  - planning is incremental
  - Can change plans as per changes in customer requirements.
- In practice, **most practical processes include elements of both plan-driven and agile approaches**.
- There are **no right or wrong software processes**.



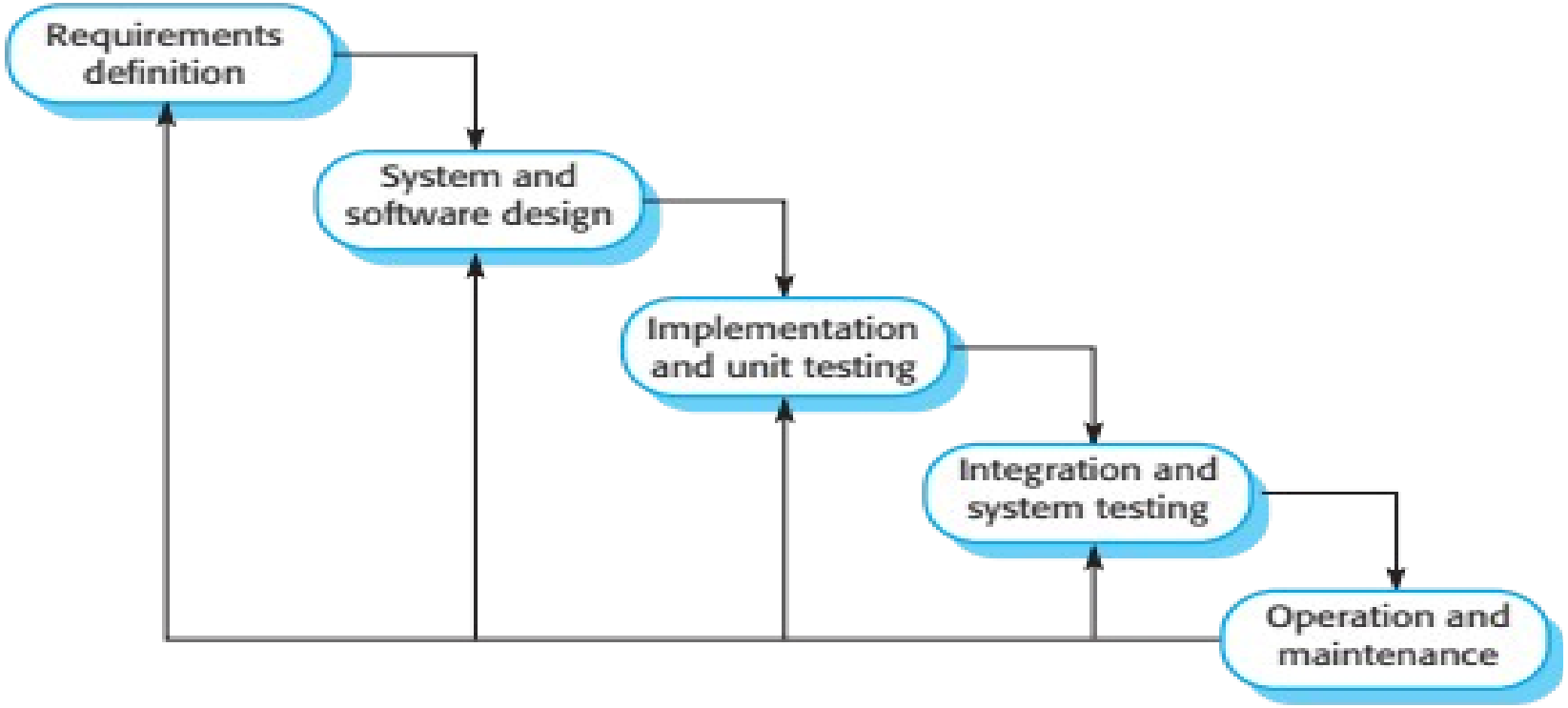
Software process models

Software process models



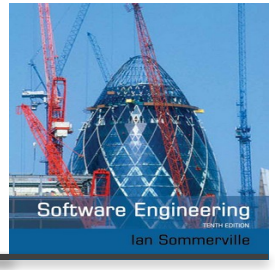
- The waterfall model
  - Plan-driven model.
  - Separate and distinct phases of specification and development.
- Incremental development
  - Specification, development and validation are interleaved.
  - May be plan-driven or agile.
- Integration and configuration
  - The system is assembled from existing configurable components.
  - May be plan-driven or agile.
- Practically, most large systems are developed using a process that incorporates elements from all these models.

The waterfall model



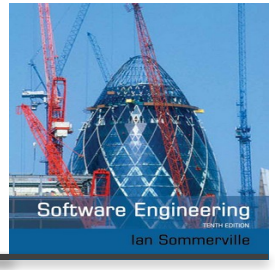
In principle, a **phase must be complete before moving onto the next phase**.

Waterfall model phases



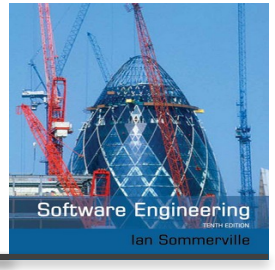
- **phases in the waterfall model:**
  - Requirements analysis and definition
  - System and software design
  - Implementation and unit testing
  - Integration and system testing
  - Operation and maintenance
- **Drawback:**
  - **intolerance to change in requirements** after the process is underway.

Waterfall model problems



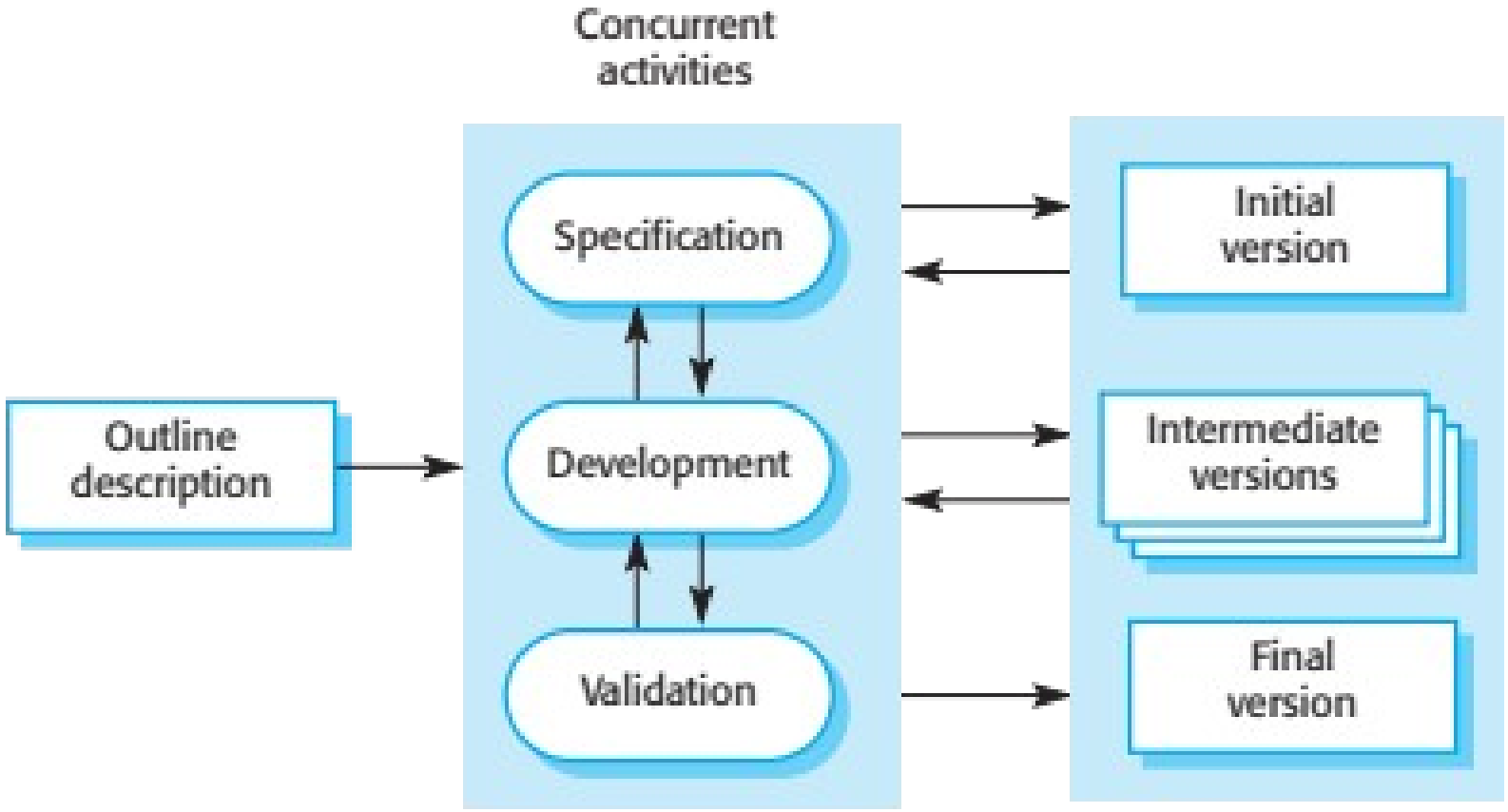
- Clearly defined requirements
- Limited to large systems.

Waterfall model problems

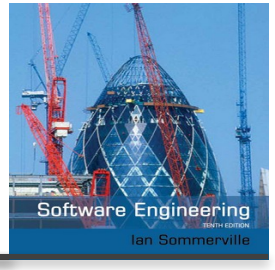


- Inflexible partitioning of the project into distinct stages makes it difficult to respond to changing customer requirements.
  - Therefore, **this model is only appropriate when the requirements are well-defined, and changes will be fairly limited during the design process.**
    - **stable requirements are very rare**.
- The waterfall model is **mostly used for large systems engineering projects where a system is developed at several sites.**
  - In those circumstances, the **plan-driven nature** of the waterfall model **makes the coordination among members easy**.

Incremental development

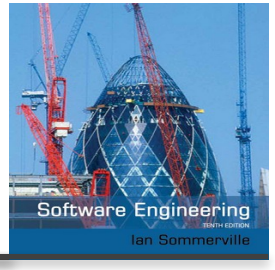


Incremental development benefits



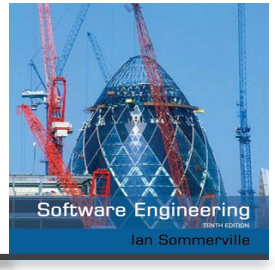
- Reduced changes' Cost
- Customer feedback ease
- Rapid delivery & deployment of useful software
- The process is not visible.
- System structure tends to degrade as new increments are added.

Incremental development benefits



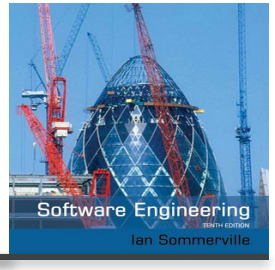
- **Reduced changes' Cost**
  - The **cost of accommodating changing customer requirements is reduced**.
  - The amount of analysis and documentation that has to be redone is much less than is required with the waterfall model.
- **Customer feedback ease**
  - It is **easier to get customer feedback on the development work** that has been done.
  - Customers can comment on demonstrations of the software and see how much has been implemented.
- **Rapid delivery & deployment of the useful software**
  - More **rapid delivery and deployment of useful software** to the customer is possible.
  - Customers are able to use and gain value from the software earlier than is possible with a waterfall process.

Incremental development problems



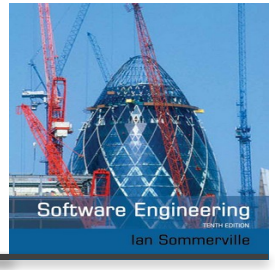
- **The process is not visible**.
  - **Managers need regular deliverables to measure progress**.
  - For deploying system quickly, mostly the documentation is not done to reduce development cost. So, in the end no documentation is present w.r.t each version.
- **System structure tends to degrade as new increments are added.**
  - Frequent change in requirements tends system to degrade as **refactoring is needed after each increment to system.**
  - Incorporating multiple software changes becomes increasingly difficult and costly.

Integration and configuration



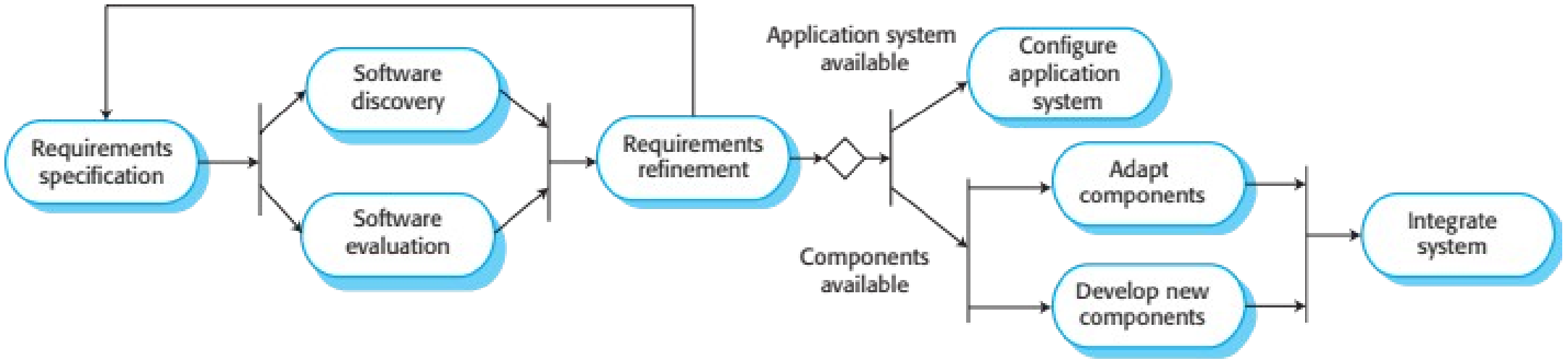
- Based on **software reuse**
- Systems are integrated from existing components or application systems (sometimes called COTS -Commercial-off-the-shelf systems).
- Reused elements may be configured to adapt their behaviour and functionality to a user's requirements
- Reuse is now the standard approach for building many types of business systems.

Types of reusable software

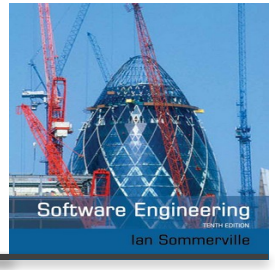


- **Stand-alone application systems** (sometimes called COTS) that are configured for use in a particular environment.
- A **package** to be **integrated with** some **framework** like .NET
- **Web services** that are developed according to service standards and which are **available for remote invocation.**

Reuse-oriented software engineering

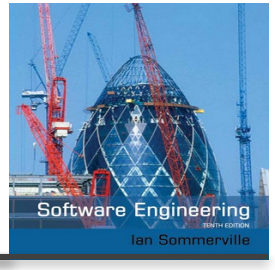


Key process stages



- Requirements’ specification
- Software discovery and evaluation
- Requirements’ refinement
- Application system configuration
- Component adaptation and integration

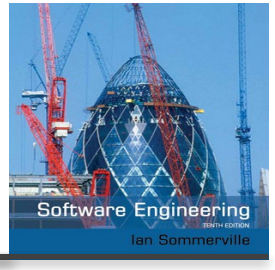
Advantages and disadvantages



Advantages	Disadvantages
Reduced costs and risks as less software is developed from scratch	But requirements compromises are inevitable so it might be possible that the system may not meet the real needs of client
Faster delivery and deployment of the system	Loss of control over the evolution of reused system elements

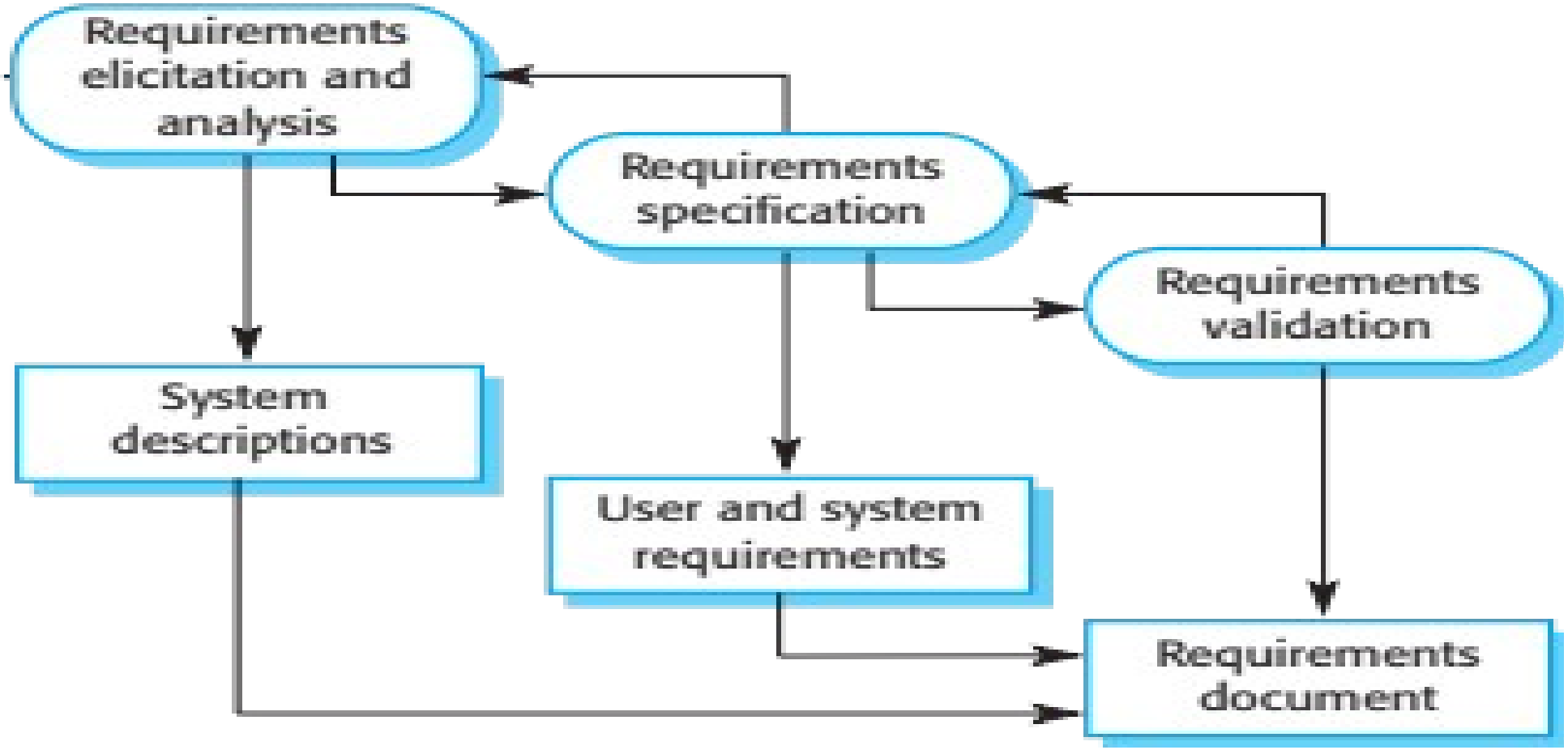
Process activities

Process activities

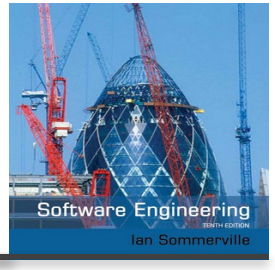


- The **four basic process activities of specification**, **development, validation** and **evolution** are organized differently in different development processes.
- For example,
  - **waterfall model ->** process activities are **organized in sequence**
  - **incremental development model** -> interleaved.

The requirements engineering process

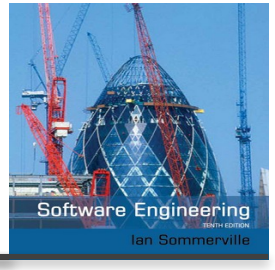


Software specification



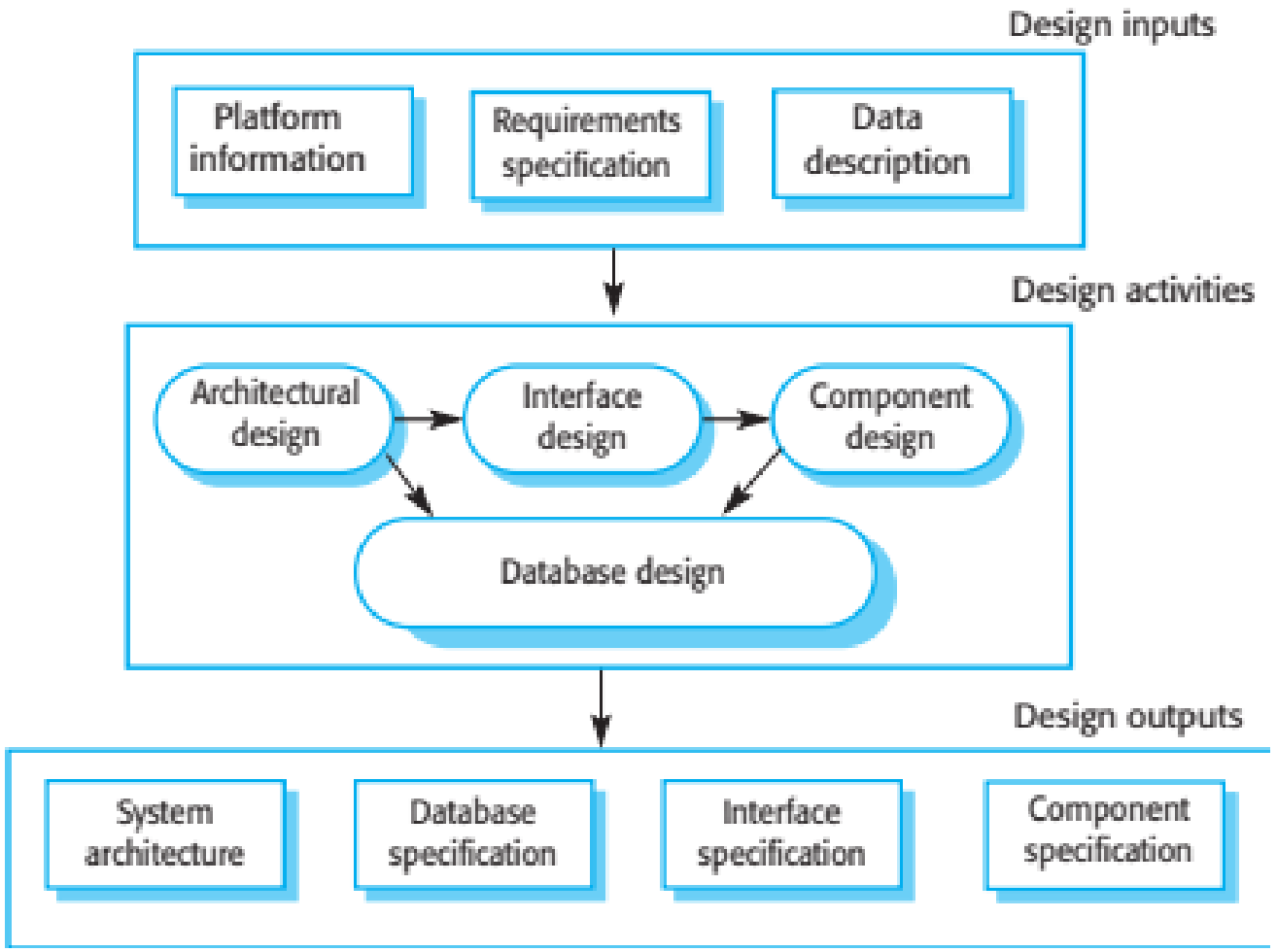
- Its a process of establishing:
  - **what services are required** and
  - **the constraints on the system's operation and development.**
- Requirements engineering process
  - Requirements elicitation and analysis
    - What do the system stakeholders require or **expect from the system**?
  - Requirements specification
    - **Defining the requirements in detail**
  - Requirements validation
    - **Whether the requirement is valid or realistic?**

Software design and implementation

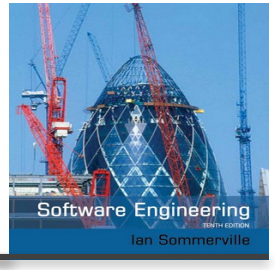


- It's a process **of converting the system specification into an executable system**.
- Software design
  - **Design a software structure that confirms the specification**;
- Implementation
  - **Translate this structure into an executable program**;
- **design and implementation activities are closely related** and may be **inter-leaved.**
- **Design may vary from system to system**.
  - For example, As in real-time systems, the database might be not needed, instead a timing module is required to minimize delay, .

A general model of the design process

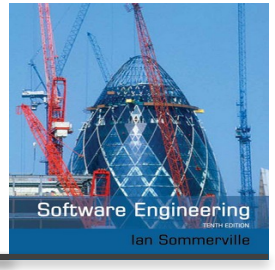


Design Inputs



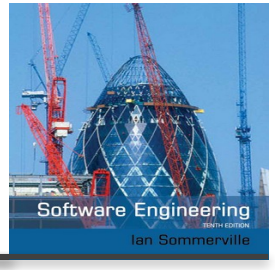
- Inputs given to system.
- That could be any **platform** related information like OS , middleware. And any other application systems if needed.
- **Data descriptions** include the data that is to be entered to the system. What could be the appropriate form and so on.
- All the specifications mentioned in requirements elicitation phase are the requirements here i.e. the ultimate **functional requirements of software**.

Design activities



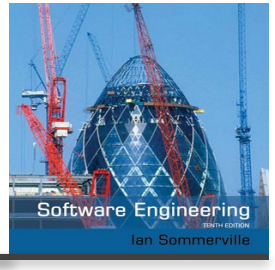
- **Architectural design**
  - identify the overall structure of the system,
  - the principal components (subsystems or modules), their relationships and how they are distributed.
- **Database design**
  - design the system data structures
  - how these are to be represented in a database.
  - Decision related to creation of a new database or reuse of an existing database?

Design activities



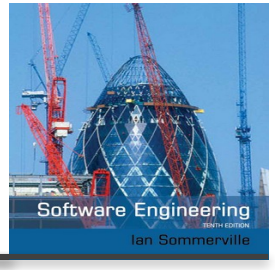
- **Interface design**
  - define the interfaces between system components.
  - Must encapsulate all the complexities
  - Should be user friendly
- **Component selection and design**,
  - you search for reusable components.
  - If unavailable, then you design the component from scratch.
  - Includes the list of changes to be done in the off the shelf component
  - or the proper technical UML diagram (if designing from scratch)

System implementation



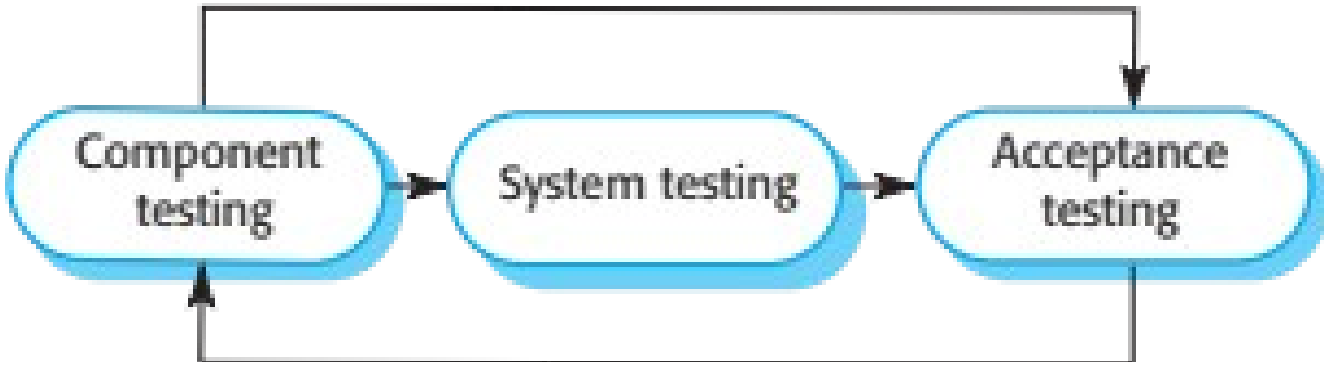
- The software is **implemented either by developing a program or programs or by configuring an application system** depending upon the scale of software application.
- **Design and implementation are interleaved activities** for most types of software system.
- **Programming** is an individual **activity with no standard process. (e.g., variable definition) ~ depends on programmer feasibility.**
- **Debugging** is the activity of finding program faults and correcting these faults.

Software validation

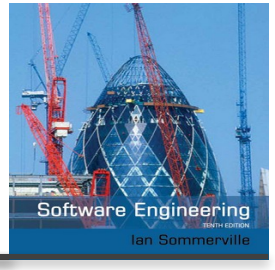


- Verification and validation (V & V) is intended to show that a **system conforms to its specification and meets the requirements of the system customer.**
- Involves system testing.
- **System testing** involves **executing the system with test cases that are derived from the specification of the real data to be processed by the system**.
- Alpha testing: testing & reviewing by the development staff
- Beta testing: testing & reviewing by the real time user

Stages of testing

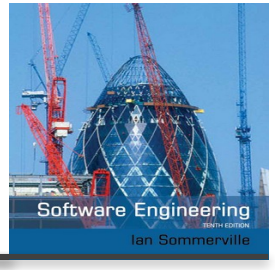


Testing stages



- Component testing
  - Component tested by developers working on the system.
  - Individual components are tested independently;
  - Components may be functions or objects or coherent groupings of these entities.
  - Test automation tools like Junit for java, VS Code plugin for JS or python could be used for unit/component testing.
- System testing
  - Testing of the system.
  - Testing of errors after different module interaction.
  - In large systems multiple subsystems are integrated first and then integrated as a final system.

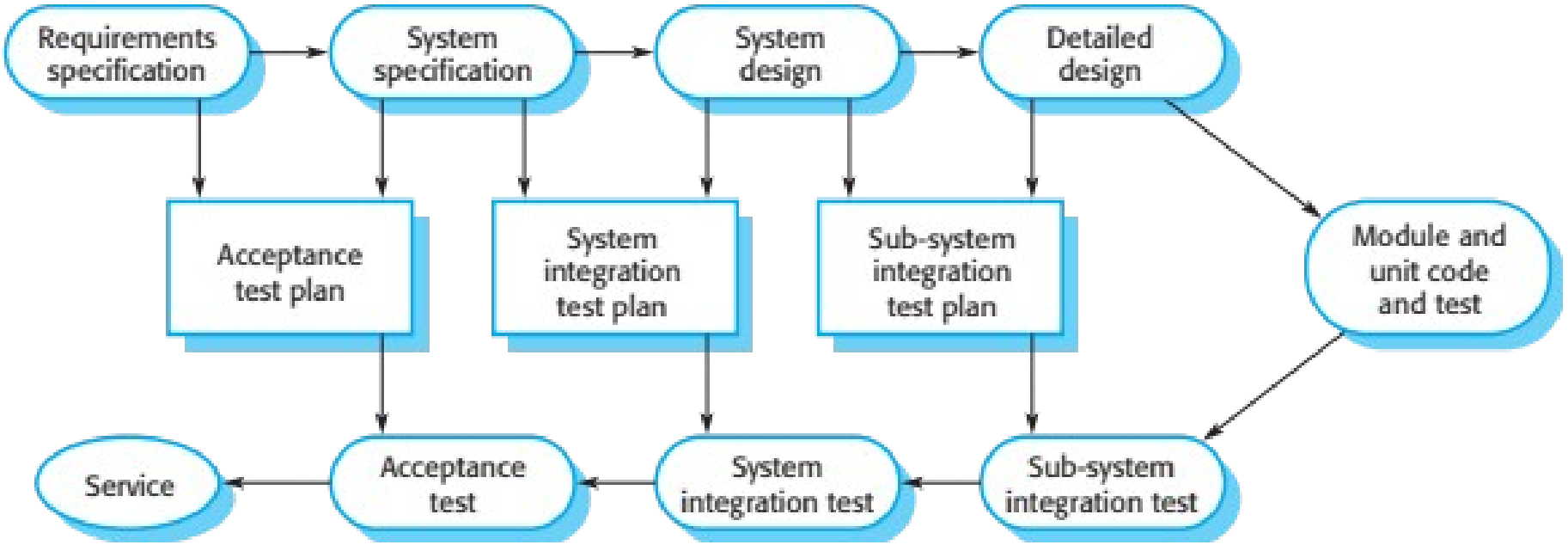
Testing stages



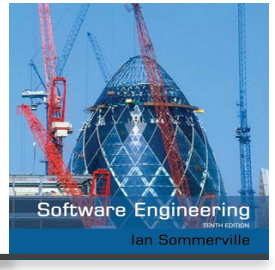
- Customer testing
  - Beta testing
  - Testing with customer data to check that the system meets the customer’s needs.
  - It might reveal error in software requirements, omission if any.

Testing phases in a plan-driven software process (V-model)

improvement of waterfall model

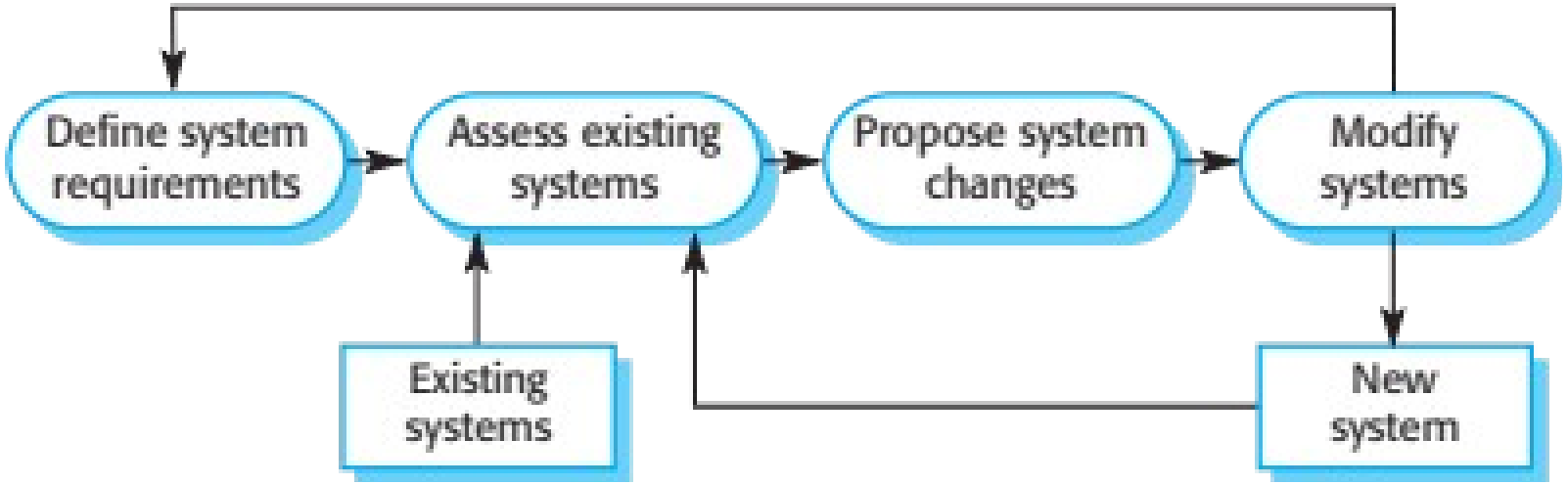


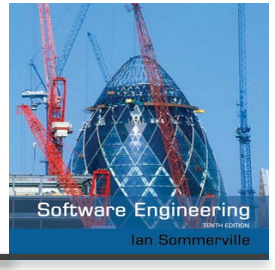
Software evolution



- Hardware changes are very expensive, as compare to that software is inherently flexible and can be modified to larger & complex systems.
- As requirements change through changing business circumstances, the software that supports the business must also evolve and change.
- Although there has been a demarcation between development and evolution (maintenance) this is increasingly irrelevant as fewer and fewer systems are completely new.

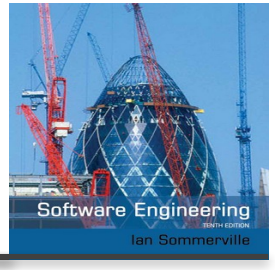
System evolution





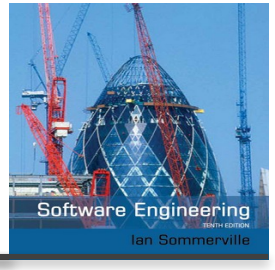
Coping with change

Coping with change



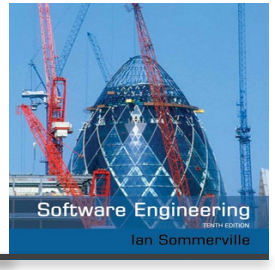
- **Change is inevitable** in all large software projects.
  - **Business changes** lead to new and changed system requirements
  - **New technologies** open up new possibilities for improving implementations
  - **Changing platforms** require application changes
- **Change leads to rework** so :
  - costs of change = cost of rework (analyzing previous req.) + cost of implementing new module

Reducing the costs of rework



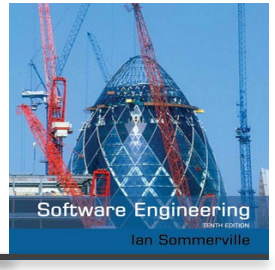
- **Proactive approach:**
  - **Change anticipation:** where the software process includes **activities that can anticipate possible changes before significant rework is required.**
- **Reactive approach:**
  - **Change tolerance**, where the process is designed so that changes can be accommodated at relatively low cost.

Coping with changing requirements



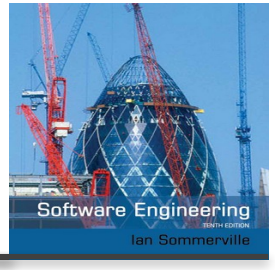
What to do when the user is not exactly sure of the requirements?

Coping with changing requirements



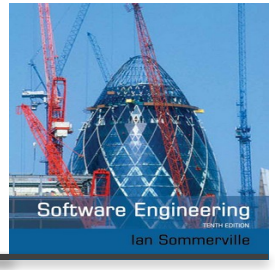
- **System prototyping,**
  - where a version of the system or part of the system is developed quickly to check the customer's requirements and the feasibility of design decisions.
  - This approach supports change anticipation.
- **Incremental delivery,**
  - where system increments are delivered to the customer for comment and experimentation.
  - This supports both change avoidance and change tolerance.

Software prototyping



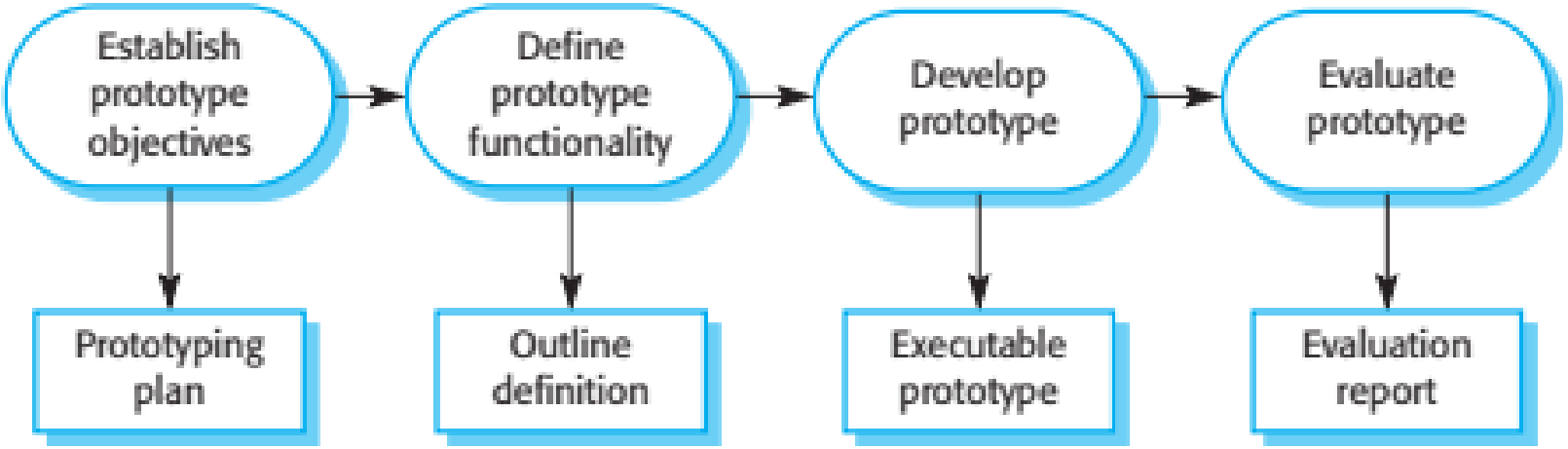
- A prototype is:
  - **an initial version of a system**
  - used to **demonstrate concepts** and
  - **Show design options**.
  - Find out about any possible **requirements' problems**
  - **Predict their solutions**
- A prototype can be used in anticipating changes in the process:
  - Used to **refine the requirements elicitation and validation phase**;
  - **In design processes** to explore options and develop a UI design;
  - **In the testing process** to run back-to-back tests.

Benefits of prototyping

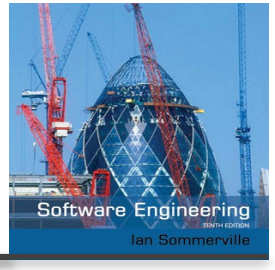


- **Improved system usability.** ~ predicts how well the system supports a no. of users.
- **A closer match to users' real needs.** ~ observe software strengths & weaknesses and then tune its requirements.
- **Improved design quality.**
- **Reduced development effort.** ~ as lesser maintenance required.
- **Improved maintainability.** ~ lesser maintenance needed because of minimal chances in requirements' falsification.

The process of prototype development

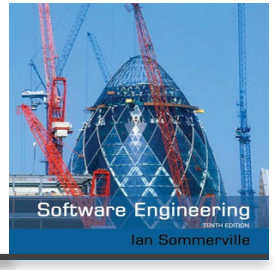


Prototype development



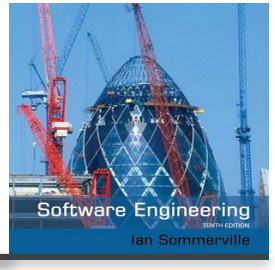
- Obviously, the prototype for the whole project is not needed.
- Correct way is to **develop prototype of the functionalities whose requirements are not clearly mentioned.**
- **Benefits:**
  - This will reduce prototyping cost
  - Accelerate development schedule
- **decide what to target:**
  - Prototype should **focus on areas of the product that are not well-understood**
  - **Focus on functional requirements like core module implementation.**
  - **Error checking and recovery may not be included** in the prototype;
  - Nonfunctional requirements like response time and memory utilization are ignored.

Prototype Development



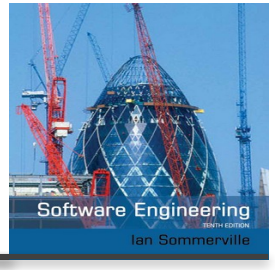
Is it good to proceed further with a prototype?

Throw-away prototypes



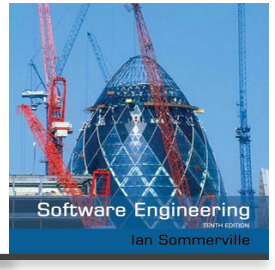
- **Prototypes should be discarded after development as they are not a good basis for a production system**
  - It may be impossible to tune the system to meet non-functional requirements;
  - Prototypes are normally undocumented;
  - The prototype structure is usually degraded through rapid change; so, it is not good to proceed further with the prototype.
  - The prototype probably will not meet normal organizational quality standards.(as it may have lesser response time, no error handling and all)

Incremental delivery



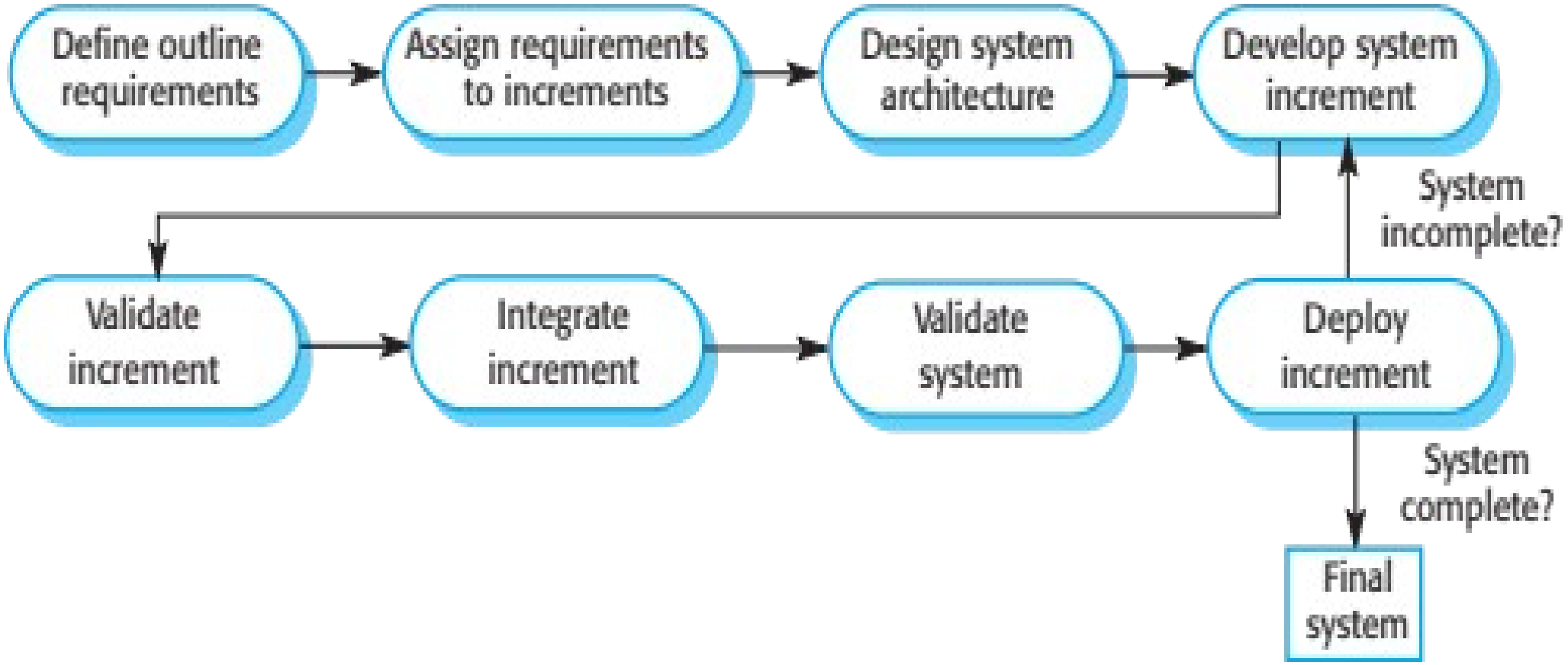
- Rather than deliver the system as a single delivery, the **software development and delivery is broken down into increments, with each increment delivering part of the required functionality.**
- **User requirements are prioritized by user himself**
- and **the highest priority requirements** are included **in early increments.**
- When an **increment is under development then its requirements are frozen.**
- though requirements for later increments can continue to evolve.

Incremental development and delivery

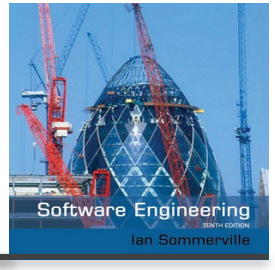


- Incremental development
  - **Develop the system in increments**
  - **and evaluate each increment before proceeding to the development of the next increment**;
  - Normal approach **used in agile methods**;
  - Evaluation is done by user/customer.
- Incremental delivery
  - **Deploy an increment for use by end-users**;
  - More **realistic evaluation about practical use of software**;

Incremental delivery

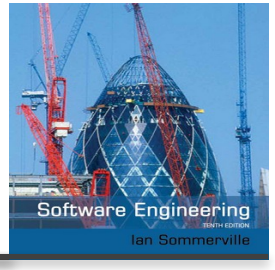


Incremental delivery advantages



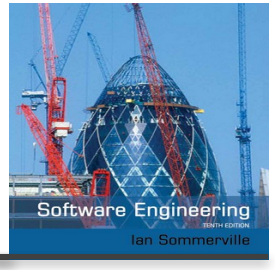
- **Early increments act as a prototype to help elicit requirements for later increments**.
- **Lower risk of** overall **project failure**.
- **The highest priority system services tend to receive the most testing**. As the first increment is a highly prioritized system that will be tested with every later increment.

Incremental delivery problems



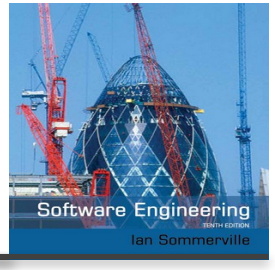
- **dependence between increments**
  - Difficult to detect as requirements are still evolving

- **Developing the contract for application is not possible**
  - Might lead to budgetary issues
  - Specification developed in conjunction with the software



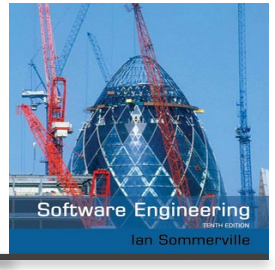
Software Process improvement

Process improvement



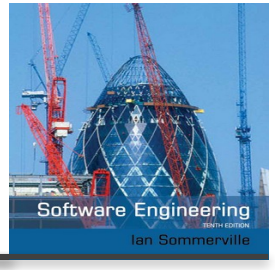
- For a newly developed Software house, With so many software rivalries in industry…. How to compete and make your place
- 3 main challenges for software houses:
  - cheaper software,
  - better software,
  - delivered with much tighter deadlines
- Solution:
  - Deliver **quality product,**
  - With **reduced costs**
  - And to **accelerate their development processes to meet deadlines**.

Process improvement



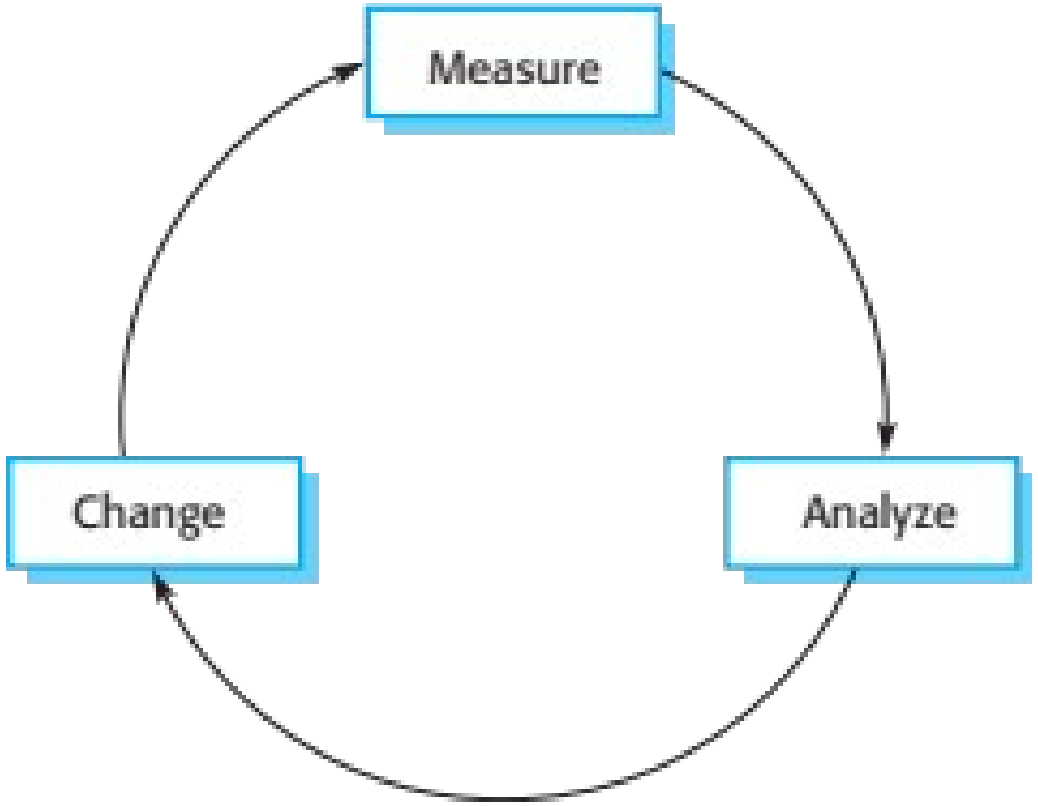
- **Software Process improvement** means:
  - **understanding existing processes**
  - and **changing these processes to increase product quality**
  - and/or **reduce costs and development time**.

Approaches to improvement



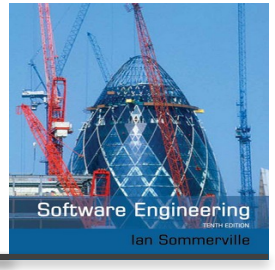
- The process maturity approach:
  - Focus on **improving process and project management**
  - and **introduces good software engineering practice** into organizations.
  - **Defined process maturity levels** to measure the reputation of an organization.
  - Higher the level of process maturity **=** good technical and management practice adopted for software development processes.
  - Goals:
    - Improve product quality & process predictability
- Generally, for plan driven approaches

The process improvement cycle



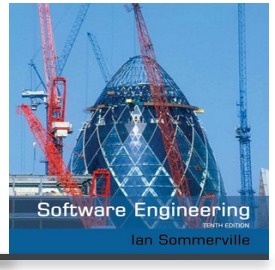
Measure process -> check if improvement would be feasible -> do the change -> measure the impact of changes on process

Process improvement activities



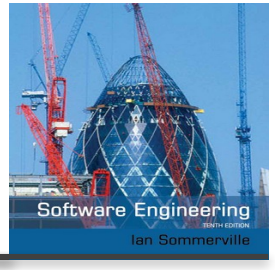
- *Process measurement (identify what to change)*
  - measure one/more attributes of the software process or product->
  - Check if process improvement have been effective -> do improvement -> remeasure the impact after improvement
- *Process analysis* (assess weakness & strengths)
  - The current **process is assessed**, and **process weaknesses and bottlenecks are identified.**
  - Process models (sometimes called process maps) that describe the process may be developed.
- *Process change(change the process)*
  - Process changes are proposed to address some of the identified process weaknesses.
  - These are introduced and the cycle resumes to collect data about the effectiveness of the changes.

Process measurement



- Wherever possible, quantitative process data should be collected
  - However, where organisations do not have clearly defined process standards this is very difficult as you don’t know what to measure. A process may have to be defined before any measurement is possible.
- Process measurements should be used to assess process improvements
  - But this does not mean that measurements should drive the improvements. The improvement driver should be the organizational objectives.

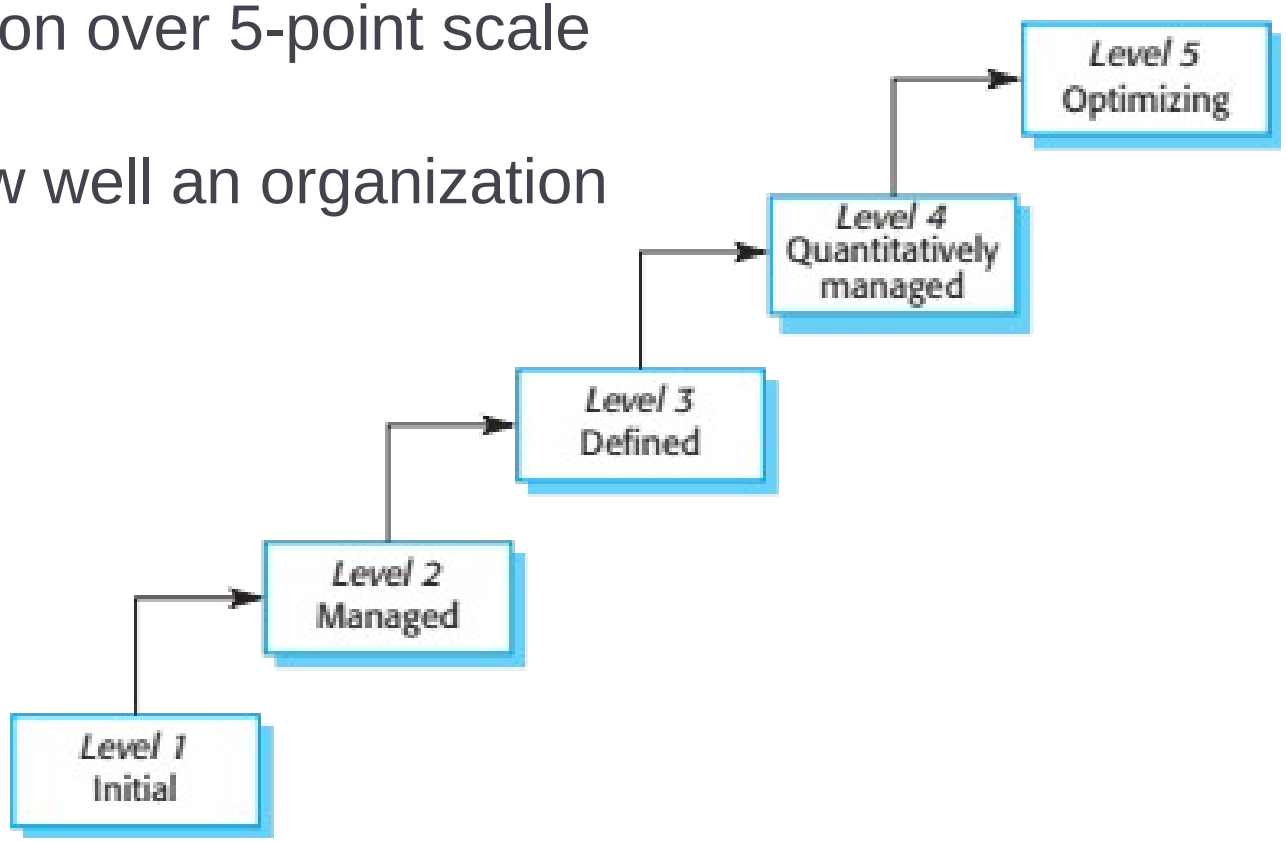
Process metrics



- Time taken for process activities to be completed
  - E.g. Calendar time or effort to complete an activity or process.
- Resources required for processes or activities
  - E.g. Total effort in person-days.
- Number of occurrences of a particular event
  - E.g. Number of defects discovered.

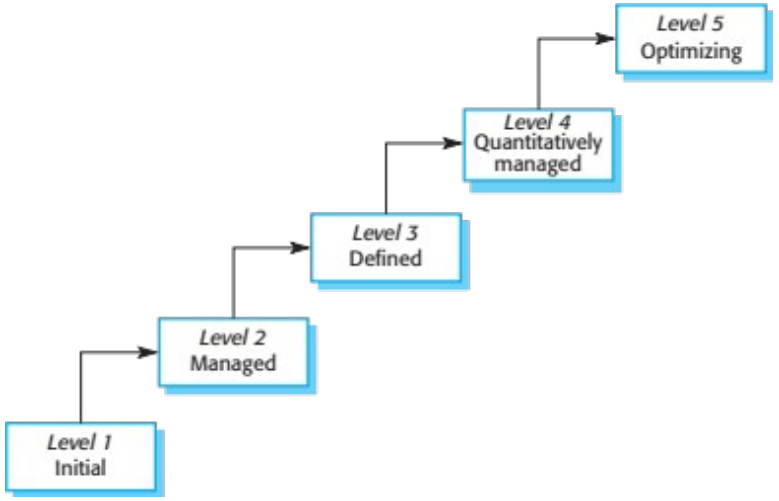
The SEI Capability maturity levels

- Rates the maturity of an organization over 5-point scale
- Not a software process model.
- Provides a mean of measuring how well an organization manages to accomplish the tasks.



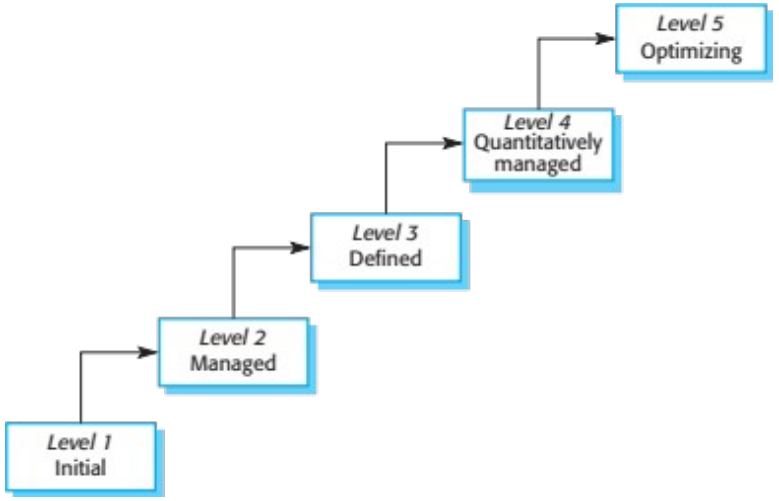
CMM initial stage

- Initial
  - Generally, the starting point of any software organization
  - Unmanaged work
  - No proper software documentation
  - No effective project management plans (generally, opt build and fix models)
  - No cost estimation
  - Vast majority of the organizations are at this level
  - Challenges:
    - Project management, planning and SQA

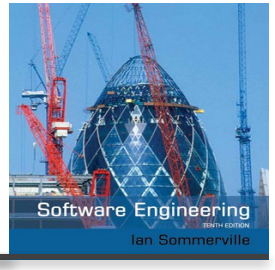


CMM Managed stage

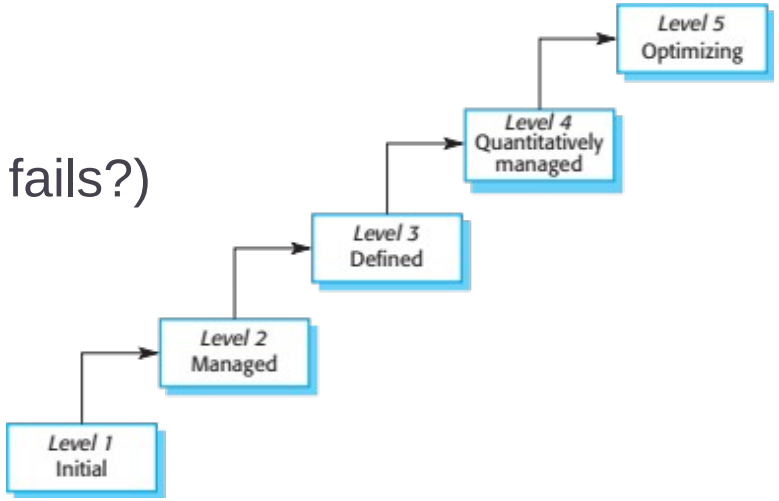
- Managed
  - Goals associated with project are satisfied
  - Defined organizational policies (when which process to be used?)
  - Documented project plan with goals
  - Resource management is properly done. (technology relevant task force)
  - Generally working on same types of software



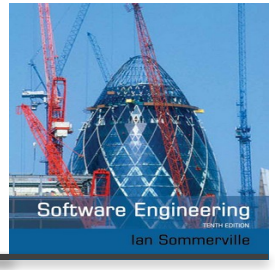
The SEI capability maturity model



- Defined
  - Process management procedures and strategies defined and used
  - Proper teamwork , teams created and hierarchy maintained
  - Collaborative learning environment
- Quantitatively Managed
  - Assess organizational performance.
  - Better project management structures.
  - Risk analysis and proactive approaches (what if a project fails?)
- Optimising
  - Process improvement strategies defined and used.
  - Increased adaption to newer trends

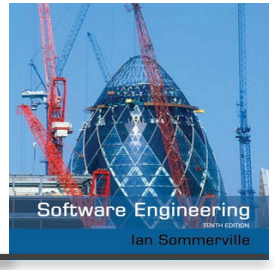


Case study 1



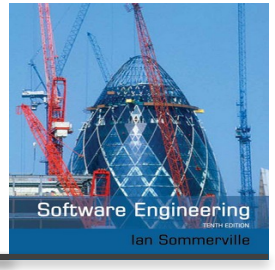
- Norton LifeLock Inc is a standalone company dedicated to consumer cyber safety. They are the pioneers of some of the security solutions in different domains like endpoint security, information security. They also have a proper team structure for managing their workflow in projects. For clear understanding of the projects, they use prototyping approaches and define the sub activities of each process as well. Moreover, for better product development, they use different techniques to appropriately predict system performance.
- CMM level is 5 (optimized) as the organization is developing innovative solutions. Moreover, they have a proper team managed working environment.

Case study 2



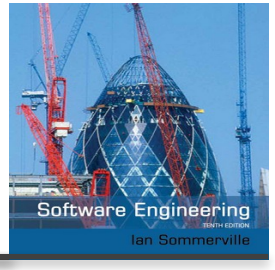
- Everyday wear is a clothing brand that wants to enter the international market. They have done with logistics and other stuff and only thing that they need is an e-commerce application. They approached an organization named as coding hub to develop an application that could run on cell phones as well as desktops. They have some incomplete set of requirements that could be provided to coding Hub employees . Since Eid holidays are around the corner and they want to deploy some essential components by November.
- Level 3 – defined : as they are working on diverse applications (desktop & mobile app)

Case study 3



- ABC Co. is an organization that designs Smart Gadgets to make life easier. They have an organized working Environment having proper teams for different kinds of projects that provide smart home solutions. For getting the customer trust, they try to accommodate the possible changes in the system as much as possible. They have also calculated the associated risk for a module and its occurrence in the system. At what level of Capability Maturity Model is ABC Co. operating. Give valid reasons for your answer.
- based on the information provided, it is likely that ABC Co. is operating at Level 4: quantitatively managed of the CMM, given their structured working environment and their ability to perform risk assessment for the project.

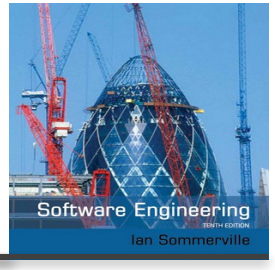
Case study 4 (identify process model)



"ABC" is a new enterprise recently started by a few young entrepreneurs, intended to develop robotic systems using IT based solutions. "TENZ" is a well-established automobile manufacturing plant, interested in automating some of the operations of the plant as first step towards automation, TENZ has decided to deploy a robotic arm to pick and place large number of bolts from one tray of a conveyer belt to another tray of a different conveyer belt. Moving one bolt at a time. TENZ has asked ABC for demonstration the robotic arm for the said purpose as a proof of concept. This is the first project being executed by ABC.TENZ would like to have a first experience of the robotic arm before signing the MOU (Memorandum of Understanding) with ABC. After MOU sign-off, TENZ would provide the complete set of requirement of the robot.

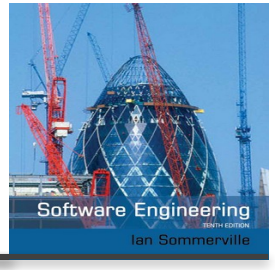
Answer: Prototyping model

Case study 5 (identify process model)



Video-Rental LTD is a small video rental store. The store lends videos to customers for a fee, and purchases its videos from a local supplier. A customer wishing to borrow a video provides the empty box of the video they desire, their membership card, and payment – payment is always with the credit card used to open the customer account. The customer then returns the video to the store after watching it. If a loaned video is overdue by a day the customer's credit card is charged, and a reminder letter is sent to them. Each day after that a further card is made, and each week a reminder letter is Sent. This continues until either the customer returns the video, or the charges are equal to the cost of replacing the video. New customers fill out a form with their personal details and credit card details, and the counter staff give the new customer a membership card. Each new customer's form is added to the customer file. The local video supplier sends a list of available titles to Video-Rental LTD, who decide whether to send them an order and payment. If an order is sent, then the supplier sends the requested videos to the store. For each new video a new stock form is completed and placed in the stock file.

Case study 5 (identify process model)



Airbus 340 flight control system is a safety critical system that implements the fly-by-wire flight system on the Airbus. The Airbus A340 is a long-haul aircraft that includes a critical fly-by-wire flight control system that mediates pilot commands and controls flight surface actuators. This is a development of the first fly-by-wire system which was introduced in the Airbus A320 aircraft.